Software Design Document

Smart Traffic Management System Using IoT

Submitted by:
Ayesha Safdar (S23BINFT1M01010)

Submitted to:
Mr. Faisal Shahzad

The Islamia University of Bahawalpur
Department of Information Technology
Faculty of Computing

Summary

This Software Design Document (SDD) describes the design and architecture of the Smart Traffic Management System (STMS) using IoT. The system is designed to optimize traffic flow, reduce urban congestion, and enhance road safety by leveraging Internet of Things technologies including sensors, cameras, communication networks, and data analytics.

The design follows an MVC (Model-View-Controller) architectural pattern deployed across edge and cloud computing layers. Key design considerations include real-time data processing with sub-2-second latency, fault-tolerant IoT sensor integration, secure encrypted communication, and a scalable microservices-based backend. The system comprises a mobile application for commuters, a web dashboard for traffic authorities, and an automated signal control subsystem — all connected through a centralized cloud platform.

Table of Contents

1. Application Architecture

The Smart Traffic Management System (STMS) follows a layered MVC (Model-View-Controller) architecture combined with a microservices approach. The system is distributed across three tiers: IoT Edge Layer, Backend Cloud Layer, and Presentation Layer.

1.1 Architectural Overview

The overall architecture consists of the following layers:

- IoT Edge Layer: Sensors, cameras, microcontrollers (Arduino/Raspberry Pi/ESP32) that collect and pre-process data at intersections.
- Communication Layer: MQTT protocol over Wi-Fi/4G/5G/LoRa for transmitting data from edge devices to the cloud.
- Backend Cloud Layer: Central server running Node.js/Python microservices for data processing, analytics, and signal control logic.
- Database Layer: MongoDB for time-series traffic data; MySQL for relational data (users, reports, configurations).
- Presentation Layer: React.js web dashboard for authorities; React Native mobile app for commuters.

1.2 MVC Components

1.2.1 Models

Model Name	Description
Traffic Data	Stores vehicle count, speed, density from IoT sensors per intersection
Signal	Represents a traffic signal with state (Red/Yellow/Green) and timing
Intersection	Represents a physical intersection with GPS coordinates and linked signals
Emergency Vehicle	Stores RFID/GPS data for emergency vehicles and preemption requests
User	System users with roles (admin, authority, driver)
Alert	Traffic alerts and notifications sent to users
Historical Report	Aggregated traffic reports for analytics

1.2.2 Controllers

The following controllers form the core logic of the system:

- Traffic Controller – manages real-time traffic data ingestion from IoT sensors
- Signal Controller – dynamically adjusts signal timing based on traffic density
- Emergency Controller – detects emergency vehicles and triggers signal preemption

- User Controller – handles authentication, authorization, and role management
- Alert Controller – generates and dispatches alerts to mobile users and dashboards
- Report Controller – generates periodic traffic reports (daily/weekly/monthly)
- Override Controller – handles manual override requests from traffic authorities

1.2.3 Controller Functions

Controller	Function	Description
Traffic Controller	Collect Data(sensor Id)	Receives real-time data from a specific IoT sensor
Traffic Controller	Analyze Flow(intersection Id)	Analyzes traffic density at a given intersection
Signal Controller	Adjust Timing(intersection Id, density)	Dynamically sets signal green/red duration
Signal Controller	Set Signal State (signal Id, state)	Manually or automatically sets a signal state
Emergency Controller	Detect Vehicle(rf id Tag)	Identifies emergency vehicle via RFID
Emergency Controller	Preempt Signal(intersection Id)	Clears signals for emergency vehicle path
User Controller	Sign Up(user Data)	Registers a new user with role assignment
User Controller	login(credentials)	Authenticates user and returns JWT token
Alert Controller	Send Alert(user Id, message)	Pushes notification to user mobile/web
Alert Controller	Broad cast Congestion(area Id)	Sends congestion alerts to all users in area
Report Controller	Generate Report(type, date Range)	Produces traffic analytics report
Override Controller	Activate Override(authority Id, signal Id)	Enables manual control by authority

1.3 External APIs and Libraries

API / Library	Purpose
Google Maps API	Map visualization of intersections and live traffic overlays
Firebase Cloud Messaging (FCM)	Push notifications to mobile app users
AWS IoT Core / MQTT Broker	Secure message queue for IoT device communication
Open Weather Map API	Environmental and weather data for traffic condition correlation
TensorFlow Lite	On-device AI inference for vehicle counting via camera feed
JWT (js on web token)	Secure token-based authentication for all API endpoints

1.4 UML Diagrams

1.4.1 System Architecture Diagram

The system operates in a three-tier architecture: IoT sensors collect data at the edge, transmit via MQTT to the cloud backend, which processes data and sends commands back to signal controllers while simultaneously updating the dashboard and mobile app.

1.4.2 Sequence Diagram – Dynamic Signal Control

Step	Actor	Action
1	IoT Sensor	Detects vehicle density and sends data via MQTT
2	MQTT Broker	Forwards message to Traffic Controller
3	Traffic Controller	Analyze Flow(intersection Id) — evaluates congestion
4	Signal Controller	Adjust Timing(intersection Id, density) — calculates new timing
5	Signal Actuator	Receives command and changes signal state
6	Dashboard	Updates real-time display of signal states and traffic map
7	Alert Controller	If congestion threshold exceeded, triggers broadcast Congestion()

1.4.3 Sequence Diagram – Emergency Vehicle Preemption

Step	Actor	Action
1	Emergency Vehicle	RFID tag detected by intersection reader
2	Emergency Controller	Detect Vehicle(rf id Tag) — identifies vehicle type
3	Emergency Controller	Preempt Signal(intersection Id) — prioritizes path
4	Signa lController	Set Signal State(signal Id, 'GREEN') for emergency path
5	Signa lController	Set Signal State (crossing Signals, 'RED') for cross traffic
6	Alert Controller	Send Alert() — notifies nearby commuters of delay

1.5 Coding Conventions

- Language: Python (backend/AI), JavaScript/Node.js (server), C++ (microcontroller), React.js (web), React Native (mobile)
- Naming: camelCase for variables and functions; Pascal Case for classes; UPPER_SNAKE_CASE for constants
- REST API endpoints follow RESTful conventions: GET / api /v1/traffic, POST /api/v1/signals
- All IoT messages follow JSON schema: { sensor Id, timestamp, vehicle Count, speed, density }
- Code is versioned using Git with feature-branch workflow and pull request reviews
- All API calls are authenticated via JWT Bearer tokens

---

2. Data Model Schema

The system uses a hybrid database approach: MongoDB for time-series IoT data (flexible schema) and MySQL for structured relational data. Below is the detailed schema for each entity.

2.1 Entity: Traffic Data

Field	Type	Description	Constraints
_id	Object Id	Unique identifier	Primary Key, Auto-generated
Sensor Id	String	Identifier of the IoT sensor	Required, Foreign Key → Sensor
Intersection Id	String	ID of the intersection	Required, Foreign Key → Intersection
timestamp	Date Time	Time of data collection	Required, Indexed
Vehicle Count	Integer	Number of vehicles detected	Required, >= 0
Avg Speed	Float	Average vehicle speed (km/h)	Required, >= 0
density	Enum	LOW / MEDIUM / HIGH / CRITICAL	Required
Congestion Level	Float	0.0 to 1.0 score	Required

2.2 Entity: Signal

Field	Type	Description	Constraints
Signal Id	INT	Unique signal identifier	Primary Key, Auto-increment
Intersection Id	INT	Linked intersection	Foreign Key → Intersection
direction	Enum	NORTH / SOUTH / EAST / WEST	Required
Current State	Enum	RED / YELLOW / GREEN	Required
Green Duration	INT	Current green phase duration (seconds)	Default: 30
Red Duration	INT	Current red phase duration (seconds)	Default: 30
Last Updated	Date Time	Timestamp of last state change	Auto-updated
Is Overridden	Boolean	Manual override active	Default: false
Override By	INT	User ID of override authority	Nullable, FK → User

2.3 Entity: Intersection

Field	Type	Description	Constraints
Inter sectionId	INT	Unique identifier	Primary Key
name	String	Location name	Required, max 100 chars
latitude	Float	GPS latitude	Required
longitude	Float	GPS longitude	Required
city	String	City name	Required
Is Active	Boolean	Operational status	Default: true
Installed Date	Date	Date IoT equipment installed	Required

2.4 Entity: User

Field	Type	Description	Constraints
User Id	INT	Unique identifier	Primary Key, Auto-increment
name	String	Full name (per ID card)	Required, max 100 chars
email	String	Email address	Required, Unique, Valid format
password	String	Hashed password	Required, min 8 chars with number
phone	String	WhatsApp-enabled phone	Required, 11 digits
role	Enum	ADMIN / AUTHORITY / DRIVER / ANALYST	Required
address	String	Residential address	Required
Display Picture	String	File path to profile image	Nullable
Is Approved	Boolean	Superuser approval status	Default: false
Created At	Date Time	Registration timestamp	Auto-generated

2.5 Entity: Emergency Vehicle

Field	Type	Description	Constraints
Vehicle Id	INT	Unique identifier	Primary Key
Rf id Tag	String	RFID identifier tag	Required, Unique
Vehicle Type	Enum	AMBULANCE / FIRE / POLICE	Required
Registration No	String	Vehicle registration number	Required, Unique
department	String	Owning emergency department	Required

2.6 Entity: Alert

Field	Type	Description	Constraints
Alert Id	INT	Unique identifier	Primary Key
Intersection Id	INT	Source intersection	FK → Intersection
Alert Type	Enum	CONGESTION / ACCIDENT / EMERGENCY / ROUTE	Required
message	String	Alert message text	Required, max 500 chars
severity	Enum	LOW / MEDIUM / HIGH / CRITICAL	Required
Sent At	Date Time	Alert dispatch timestamp	Auto-generated
Target Users	String	User IDs or broadcast area	Required

2.7 ER Diagram Description

The key relationships between entities are:

- Intersection (1) — (N) Signal: One intersection has multiple signals (one per direction)
- Intersection (1) — (N) Traffic Data: One intersection generates many traffic data records over time
- Signal (1) — (N) Traffic Data: Each signal's timing changes are linked to traffic data readings
- User (1) — (N) Alert: An authority user can trigger multiple alerts

- Emergency Vehicle (1) — (N) Alert: Emergency vehicle detection triggers preemption alerts
- User (M) — (N) Intersection: Traffic authorities can manage multiple intersections

---

3. User Interface

3.1 Design Philosophy

- Color Scheme: Primary blue (#1F4E79, #2E75B6), white backgrounds, red/amber/green for signal states
- Typography: Arial / Roboto, 14px body, 18–24px headings, high contrast for readability
- Design Niche: Functional, data-dense dashboard for authorities; clean minimal interface for commuters
- Responsiveness: Web dashboard targets 1280px+ desktop screens; mobile app targets 360px–414px portrait
- Accessibility: WCAG 2.1 AA compliant; high contrast ratios; touch-friendly tap targets (48x48px min)

3.2 Screens / Pages

3.2.1 Screen: Login / Sign Up

Purpose: Authenticate existing users or register new users pending admin approval.

Control	Type	Input Constraint	Action
Name	Text Box	Required, max 100 chars	Stores full name
Email	Text Box	Required, valid email format	Used as username
Password	Password Box	Min 8 chars, must include number	Stored as bcrypt hash
Phone	Text Box	Required, exactly 11 digits	WhatsApp-enabled number
Address	Text Box	Required, max 200 chars	Residential address
Display Picture	File Upload	JPG/PNG, max 2MB	Profile image
Sign Up Button	Button	All fields validated before submit	Submits registration request
Login Button	Button	Email + Password required	Authenticates user, returns JWT

3.2.2 Screen: Admin Dashboard

Purpose: Central control panel for traffic authorities to monitor all intersections in real-time.

Control	Type	Input Constraint	Action
Live Traffic Map	Map Component	Read-only, zoom/pan	Displays all intersections with color-coded density
Intersection Selector	Dropdown	Required to view details	Filters data to selected intersection
Signal State Panel	Status Grid	Read-only (auto), editable (override)	Shows Red/Yellow/Green per direction
Manual Override Toggle	Toggle Switch	Authority role required	Activates Over ride Controller
Set Green Duration	Number Input	10–120 seconds, integer	Calls Signal Controller .adjust Timing()
Active Alerts Panel	Alert List	Read-only, sortable	Lists current congestion/emergency alerts
Generate Report	Button + Date Picker	Date range required	Calls Report Controller. Generate Report()

3.2.3 Screen: Traffic Analytics & Reports

Purpose: Visualize historical traffic data and export reports for urban planning.

Control	Type	Input Constraint	Action
Report Type Selector	Radio Buttons	Daily / Weekly / Monthly	Sets report aggregation period
Intersection Filter	Multi-select Dropdown	Optional, all if empty	Filters report by location
Date Range Picker	Date Range Input	Start <= End, not future	Defines analysis window
Traffic Flow Chart	Line Chart	Read-only visualization	Shows vehicle count over time
Peak Hours Chart	Bar Chart	Read-only visualization	Highlights busiest time slots
Export Button	Button	PDF or CSV format	Downloads generated report

3.2.4 Screen: Mobile App – Commuter View

Purpose: Provide real-time traffic information and route suggestions to drivers.

Control	Type	Input Constraint	Action
Map View	Map Component	Current location required (GPS)	Displays live traffic density around user
Destination Input	Search Box	Required, valid address	Triggers route calculation via Google Maps API
Alert Notifications	Push Notification / Banner	Auto-generated by system	Notifies of congestion, accidents, emergencies
Alternative Route Button	Button	Active when congestion detected	Calls Alert Controller. Send Alert() with route
Parking Availability	Icon Grid	Read-only, location-based	Shows available spots near destination
Notification Settings	Toggle List	User preference	Enables/disables specific alert types

3.2.5 Screen: User Management (Admin Only)

Purpose: Allows system administrators to approve, manage, and assign roles to users.

Control	Type	Input Constraint	Action
Pending Approvals List	Data Table	Read-only, paginated	Lists users awaiting approval
Approve Button	Button	Admin role required	Sets isApproved = true, notifies user
Reject Button	Button	Admin role required	Deletes registration, notifies user
Role Assignment Dropdown	Dropdown	ADMIN/AUTHORITY/DRIVER/ANALYST	Updates user role in database
Search/Filter Bar	Text Box + Dropdown	Optional	Filters users by name, role, or status

3.3 Input Constraints Summary

Field	Constraint	Validation Rule
Email	Format validation	Must match regex: ^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$
Password	Strength validation	Minimum 8 characters, at least 1 number
Phone	Length validation	Exactly 11 digits, numeric only
Signal Duration	Range validation	Integer between 10 and 120 seconds
Date Range	Logic validation	Start date must be before or equal to end date
Vehicle Count	Non-negative	Integer >= 0
GPS Coordinates	Range validation	Latitude: -90 to 90; Longitude: -180 to 180

---

4. References

Authors should provide references in IEEE format.

- [1] R. T. Naik et al., "Smart Traffic System using IoT," International Journal of Engineering Research, 2020.
- [2] IEEE, "IoT-based Smart Traffic Signal Monitoring System," IEEE Transactions on Intelligent Transportation Systems, 2021.
- [3] Cisco, "Smart+Connected Traffic Solution Overview," Cisco Systems, 2022.
- [4] Google, "Maps JavaScript API Reference," Google Developers, 2024.
- [5] MQTT.org, "MQTT Version 5.0 Specification," OASIS Standard, 2019.
- [6] A. Zanella et al., "Internet of Things for Smart Cities," IEEE Internet of Things Journal, vol. 1, no. 1, pp. 22-32, Feb. 2014.